

Sistemas Multiagentes (PPGCA-CT e PPGCC-PG)

Practical Reasoning Agents

Prof^a Celia Ghedini Ralha

Universidade Tecnológica Federal do Paraná (UTFPR)
Curitiba/PR, Brasil

09 de outubro de 2026

Agenda

- What is Practical Reasoning?
 - ▶ Components
- Planning System Illustration
 - ▶ STRIPS representation
- Implementing Practical Reasoning Agents
 - ▶ HOMER agent
- Planning Domain Definition Language (PDDL)
 - ▶ Simple Task Planning Example
 - ▶ PDDL & Alternative Language Planners

Practical Reasoning²

Distinguish practical from theoretical reasoning:

- **Theoretical** reasoning is directed towards **beliefs**
- **Practical** reasoning is directed towards **actions**

Practical reasoning is a matter of weighing conflicting considerations for and against competing options, where the relevant considerations are provided by what the agent desires/values/cares about and what the agent believes.

Michael Bratman, Intentions in Communication (2003), MIT Press, page 17¹

¹ <https://mitpress.mit.edu/9780262517041/intentions-in-communication/>

² M. Wooldridge. An Introduction to MultiAgent Systems, John Wiley & Sons Ltd., 2nd ed., 2009 - Chap. 4

Practical Reasoning Components

Human practical reasoning consists of two activities:

- **Deliberation**

- ▶ process of establishing what state the agent is working towards
the outputs are intentions

- **Means-ends Reasoning**

- ▶ deciding on how to get to that state
the outputs are plans

Practical Reasoning Example

The design of a course of action that will achieve some desired goal

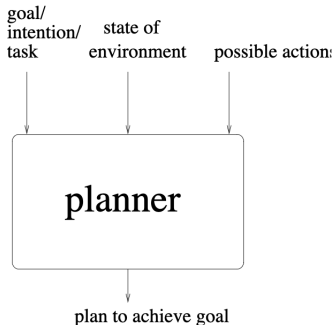


- For example, an agent playing chess must **deliberate** what to do
 - ▶ Will it be looking to defend?
 - ▶ Attack a certain piece?
 - ▶ Claim check-mate?
- once it has deliberated & picked a task to work towards, it performs **means-ends reasoning** to decide how to achieve set task
 - ▶ it may be trivial - move a particular piece to take another one, or
 - ▶ it may be more complex - a plan that will take several moves

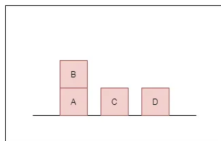
Planning System

Basic idea (representation of)

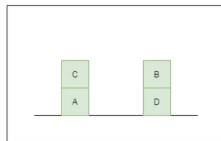
- goal/intention/task to be achieved
- state of environment
- possible actions available to agent
- plan to achieve goal



Planning System Illustration



Initial State



Goal State

- Blocks world with a robot arm, table-top, and 4 blocks (A,B,C,D)
- Initial state environment (predicates and meaning)
 - ▶ $On(B, A)$ - obj B on top of obj A
 - ▶ $OnTable(C), onTable(D)$ - obj C and D is on the table
 - ▶ $ClearC, ClearD$ - nothing on top obj C and D
- Goal state to be achieved
 - ▶ $On(C, A), On(B, D)$
 - ▶ $Clear(C), Clear(B)$
- Actions available to agent
 - ▶ $Pickup(x)$ - arm picks up obj x from the table
 - ▶ $PutDown(x)$ - arm places obj x onto the table
 - ▶ $Holding(x)$ - arm holds obj x
 - ▶ $Stack(x, y)$ - stack obj x on top of y
 - ▶ $UnStack(x, y)$ - unstack obj x over y

Use the closed world assumption: anything not stated is assumed to be false.

STRIPS Representation³

Actions in STRIPS has:

- a name – may have arguments
- a pre-condition list – list of facts which must be true for action to be executed
- a delete list – list of facts that are no longer true after action is performed
- an add list – list of facts made true by executing the action

The **stack** action - the robot arm places obj x it is holding, placed on top of object y

$$\begin{aligned} & \textit{Stack}(x,y) \\ \text{pre } & \textit{Clear}(y) \wedge \textit{Holding}(x) \\ \text{del } & \textit{Clear}(y) \wedge \textit{Holding}(x) \\ \text{add } & \textit{ArmEmpty} \wedge \textit{On}(x,y) \end{aligned}$$

The **pickup** action - the robot arm picks up obj from the table

$$\begin{aligned} & \textit{Pickup}(x) \\ \text{pre } & \textit{Clear}(x) \wedge \textit{OnTable}(x) \wedge \textit{ArmEmpty} \\ \text{del } & \textit{OnTable}(x) \wedge \textit{ArmEmpty} \\ \text{add } & \textit{Holding}(x) \end{aligned}$$

³

Stanford Research Institute Problem Solver (STRIPS) - automated planner description language by Richard Fikes & Nils Nilsson (1971), AI Center for Scientific Research Institute (SRI) International, Stanford Research Institute, California, USA.

Implementing Practical Reasoning Agents

What is a plan?

- A sequence of actions (list), with variables replaced by constants

First idea of practical reasoning agent implementation (algorithm)

```
1. while true
2.     observe the world;
3.     update internal world model;
4.     deliberate about what intention
       to achieve next;
5.     use means-ends reasoning to get
       a plan for the intention;
6.     execute the plan
7. end while
```

Problem: **deliberation** & **means-ends reasoning** processes aren't instantaneous

- they have a time cost
- the world may change

Implementing Practical Reasoning Agents

How does an agent deliberate?

- begin by trying to understand what options are available
- choose between them and commit to some

Then, chosen options are intentions. Possible algorithm:

```
1.  $B := B_0$ ; /* initial beliefs */  
2. while true do  
3.     get next percept  $\rho$ ;  
4.      $B := brf(B, \rho)$ ;  
5.      $I := deliberate(B)$ ;  
6.      $\pi := plan(B, I)$ ;  
7.      $execute(\pi)$   
8. end while
```

Controlling Intention Reconsideration

Incorporate explicit meta-level control to decide whether/not to reconsider

Algorithm: Practical Reasoning Agent Control Loop

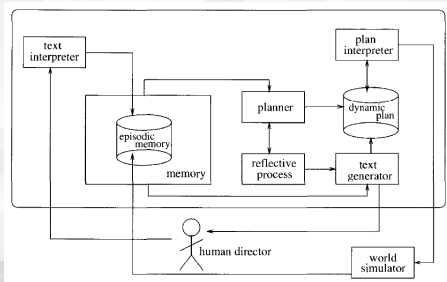
```

1.
2.  $B \leftarrow B_0$ ;      /*  $B_0$  are initial beliefs */
3.  $I \leftarrow I_0$ ;    /*  $I_0$  are initial intentions */
4. while true do
5.   get next percept  $\rho$  through see(...) function;
6.    $B \leftarrow brf(B, \rho)$ ;
7.    $D \leftarrow options(B, I)$ ;
8.    $I \leftarrow filter(B, D, I)$ ;
9.    $\pi \leftarrow plan(B, I, Ac)$ ;
10.  while not (empty( $\pi$ ) or succeeded( $I, B$ ) or impossible( $I, B$ )) do
11.     $\alpha \leftarrow head(\pi)$ ;
12.    execute( $\alpha$ );
13.     $\pi \leftarrow tail(\pi)$ ;
14.    get next percept  $\rho$  through see(...) function;
15.     $B \leftarrow brf(B, \rho)$ ;
16.    if reconsider( $I, B$ ) then
17.       $D \leftarrow options(B, I)$ ;
18.       $I \leftarrow filter(B, D, I)$ ;
19.    end-if
20.    if not sound( $\pi, I, B$ ) then
21.       $\pi \leftarrow plan(B, I, Ac)$ 
22.    end-if
23.  end-while
24. end-while
  
```

HOMER Agent that Plans⁴

Vere & Bickmore argued that the enabling technologies for intelligent agents are sufficiently developed to be able to construct a prototype autonomous agent, with linguistic ability, planning and acting capabilities.

- simulated robot submarine that exists in a two-dimensional 'Seaworld' (partial knowledge)
- takes instructions from a user in a limited subset of English with about an 800 word vocabulary
- instructions can contain moderately sophisticated temporal references
- Homer plan how to achieve its instructions (typically relate to collect and move items around the seaworld), execute its plans, modifying them as required during execution
- has limited episodic memory allowing to answer questions about past experiences



PDDL

What is PDDL?

- Planning Domain Definition Language (PDDL) standard encoding language for “classical” planning tasks
- PDDL was proposed for empirical evaluation of planner performance, development standard sets of problems in comparable notations⁵
- Languages like PDDL can describe an entire space of possible problems where agents can take the same actions, but in different environments and with different goals

Components of a PDDL planning task

- **Domain** - describe how the world works
 - ▶ **Predicates** - properties of objects (can be true or false) - have associated parameters denoted by ?
 - ▶ **Actions** - ways of changing the world's state (Move, Pick, Place)
- **Problem** - where agents can take actions
 - ▶ **Objects** - things in the world that interest
 - ▶ **Initial state** - the state of the world that we start in
 - ▶ **Goal specification** - things that we want to be true

⁵

PDDL was produced by the competition committee at 4th Int. Conf. on Artificial Intelligence Planning Systems 1998 (AIPS-98)
<https://www.cs.cmu.edu/~mmv/planning/readings/98aips-PDDL.pdf>

PDDL Simple Example⁶

Suppose a robot in a simple world. Let's command our robot to perform a task: take the apple from the shelf and put it on the table.

2 task planning fundamental pieces

Domain (task-agnostic)

Predicates: (Location ?loc) is a unary predicate, meaning it has one parameter

(Robot ?r), (Object ?o), (Location ?loc), (At ?r ?loc), (Holding ?r ?o)...

Actions: move(?r ?loc1 ?loc2), pick(?r ?o ?loc), place(?r ?o ?loc)

Problem (task-specific)

Objects: (Robot robot), (Location shelf), (Location table), (Object apple)

Initial state: (HandEmpty robot), (At robot table), (At apple shelf)

Goal specification: (At ?r ?loc) is a binary predicate with 2 parameters
(At apple table)

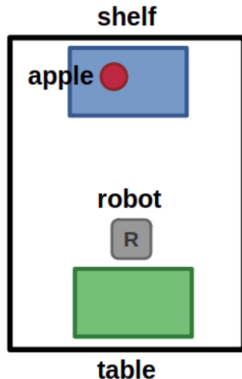
Move action definition (then define pick, place)

:action move

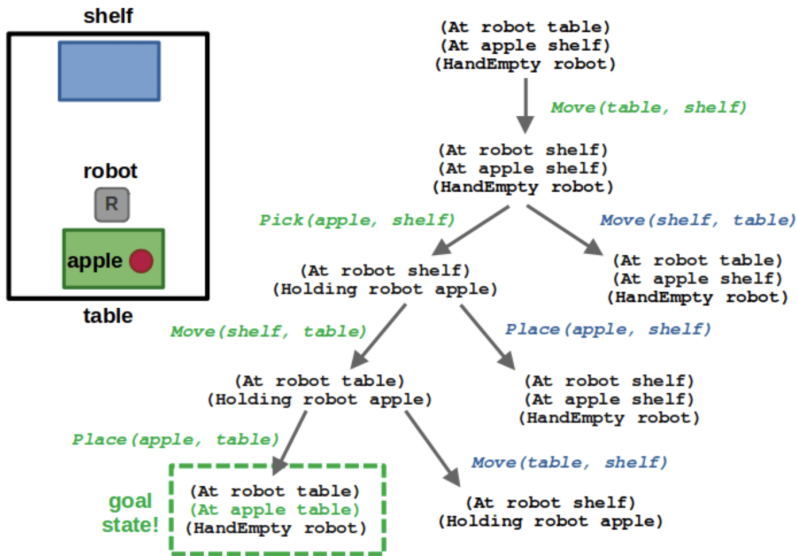
:parameters (?r ?loc1 ?loc2)

:precondition (and (Robot ?r) (Location ?loc1) (Location ?loc2) (At ?r ?loc1))

:effect (and (At ?r ?loc2) (not (At ?r ?loc1)))



PDDL Simple Example



Task Planning

- Task planning is solved using search algorithms
- There is good reason for all this overhead in defining a planning domain and a good language to express it in
- Much of the literature is about solving complex problems as efficiently as possible
- Task planning problems scale up, computational costs increase
- Complexity. Simple analogy: Imagine an enormous maze
 - ▶ NP-complete
Can you find an optimal path from A to B using at most 100 steps?
If someone gives you a possible solution, you can check in polynomial time
 - ▶ PSPACE-complete
Is there any path from A to B?
You systematically explore possible configurations (time/space), potentially requiring enormous amount of computation
- More planners need to be proposed

Planners using PDDL

Classical Planning (single agent)

- **Toplevel** - planner that works with PDDL for hierarchical planning tasks. *D. S. Weld et al. (1998) Efficient, hierarchical planning.*
- **Blackbox** - earlier, influential planner translating PDDL into Quantified Boolean Formulas (QBF) for planning. *S. Kambhampati et al. (2002) Blackbox: Planning as Satisfiability in STRIPS.*
- **Multi-modal Instantaneous Planning System (MIPS)** - fast planner supporting PDDL with temporal and numeric features. *M. Ryan & J. Hoffmann (2003) MIPS: A Fast & Robust Planner for PDDL Problems.*
- **jSHOP2** - planning system that extends SHOP2 for hierarchical planning with PDDL-like domains. *A. Botea et al. (2005) Hierarchical Planning with SHOP2.*
- **Fast-Downward** - widely used planner for classical planning problems, supporting various heuristics. *M. Helmert (2006) The Fast Downward Planning System.*
- **Local Planning Graph (LPG)** - Uses local search techniques for temporal planning in PDDL domains. *P.-L. Richard, & H. Geffner (2008) LPG: A localized planner for PDDL domains.*
- **ROLLER** - planning involving preferences, encoded in PDDL. *R. Ramakrishnan et al. (2009) Preference-based planning with ROLLER.*
- **Metric-FF** - extension of the FF planning system that handles metric information within PDDL. *J. Hoffmann & B. Nebel (2011) The FF Planning System: Fast Plan Generation Through Heuristic Search.*

Planners using Alternative Languages



1. HTN Planners (Hierarchical Task Networks)

- Use hierarchical task decompositions instead of logical predicates.
- Represent plans as methods and subtasks (often in custom languages).
- Well suited for complex, structured domains.

Examples: SHOP, SHOP2, Pyhop



2. STRIPS-like Planners (Action Language)

- Use a simpler representation based on preconditions and effects.
- Often implemented with custom languages or domain-specific formats (not PDDL).
- Efficient and widely used in classical domains.

Examples: UCPOP, STAN, NOAH



3. Answer Set Programming (ASP) Planners

- Represent planning as a logic program in ASP (e.g., using Clingo).
- Encode actions, constraints, and goals as logical rules.
- Good for rich constraints and complex reasoning.

Examples: ASP-based planners (e.g., Clingo, ASP-PLAN)



4. SAT/SMT-based Planners

- Encode planning problems as Boolean or SMT constraints.
- Use SAT/SMT solvers to find a valid plan.
- Can handle rich numeric and temporal constraints.

Examples: SatPlan, MIPS, SMTPlan



5. Probabilistic / Uncertainty-aware Planners

- Use probabilistic models or decision-theoretic formulations.
- Often based on MDPs, POMDPs or custom probabilistic languages.
- Handle uncertainty in actions and observations.

Examples: POMDP planners, PRP, PPDDL (probabilistic)



6. Domain-specific / Learning-based Planners

- Use custom representations tailored to a domain (e.g., robotics, scheduling, logistics).
- May rely on machine learning or search heuristics instead of declarative languages.
- Often integrated with domain knowledge or data.

Examples: Deep RL planners, learned planners, robotic motion planners

Planners can use hierarchical, logical (boolean, satisfiability modulo theories), constraint-based, probabilistic, domain-specific representations depending on the problem and application

Summary

- Practical Reasoning Concepts & Components
- Planning System Illustration
- STRIPS Representation
- Implementing Practical Reasoning Agents: HOMER agent
- Planning Domain Definition Language (PDDL): Simple Example
- Planners Using PDDL & Alternative Languages

Thank you for coming!



Profª Celia Ghedini Ralha

celiaghedini@utfpr.edu.br